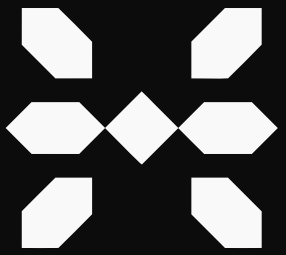
HOOX · PYNE · DOCUMENTATION



AUDITORS · CONTRIBUTORS · PARITY HUNTERS

REFERENCE MANUAL

Compatibility, status, gaps, numerical validation, pine-worker

Open-source Pine Script evaluation framework

2026-10-17 DIN A4 · HTTPS://HOOX.SH/PYNE/DOCS OPEN SOURCE

CONTENTS

01 Ecosystem

02 Commercial

03 Compatibility

04 Missing Features

05 Pine V6 Surface

06 Numerical Validation

07 Roadmap

08 Contributing

ECOSYSTEM

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Ecosystem" description: "PYNE, PyneTS, pyne-worker, pyne-agent-worker, AXIS, and HOOX — what each one is and is not."

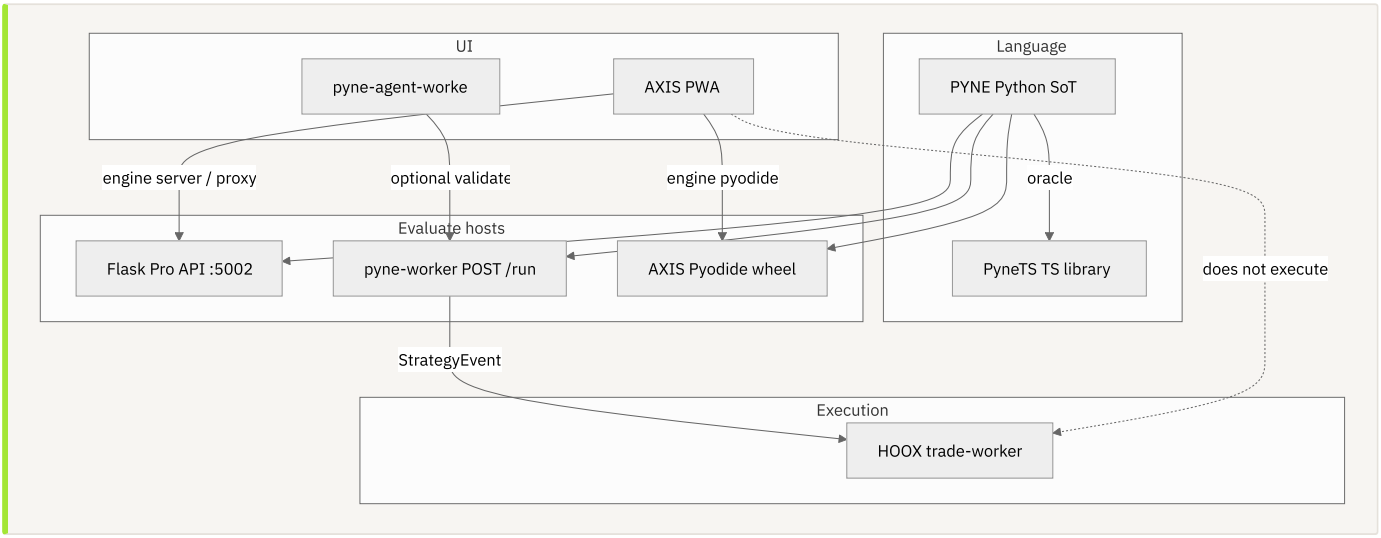
ECOSYSTEM

Abstract

The HOOX family splits **language**, **chart**, and **execution**. Mixing the names is the most common docs failure: `pyne-worker` is not `pynets`, and AXIS does not place HOOX orders.

Product	Role	Docs
PYNE (`hoox-pyne` , import `pynescript`)	Python language SoT: parse, LSP, Pro API, `Runtime.run`	this site
PyneTS (`@hoox-sh/pynets`)	TypeScript / Bun **library + CLI**	PyneTS
pyne-worker	Python Cloudflare® isolate — `POST /run`	pyne-worker · HOOX isolate
pyne-agent-worker	NL → Pine (Workers AI™)	agent · AXIS plugin
AXIS	Charting PWA (engines call Python)	AXIS
HOOX	Edge trade mesh	HOOX

Conceptual model



Invariant: evaluation never requires a proprietary chart host. AXIS is an optional chart. HOOX is an optional execution mesh. PyneTS is an optional TypeScript import.

Interface surface

Published versions (static snapshot; refresh manually with `bun run docs:versions` when a registry moves):

Checkouts (this repo vs sisters)

Checkout	What you open
hoox-sh/pyne	PYNE SoT (`hoox-pyne` **0.6.12**)
hoox-sh/pynets	Standalone `@hoox-sh/pynets` **0.5.0** (JS compile + stream)
hoox-sh/pyne-worker	Python edge `/run`
hoox-sh/pyne-agent-worker	NL authoring (**not** in this tree)
hoox-sh/axis	AXIS PWA
hoox-sh/hoox	Mesh monorepo

Evaluate contract

Flask `POST /run`, pyne-worker `POST /run`, and in-process `Runtime.run` share the evaluate contract: `script` + OHLCV + `mode` → `plots` / `events` / `alerts`. PyneTS speaks the **library** form of that envelope (`RuntimeResult`), not HTTP.

Internals

Docs are authored in product trees and synced to hoox.sh:

Tree	Public base
`pynescript/docs/pyne/**`	`/pyne/docs`
`axis/docs/**`	`/axis/docs`
`hoox/docs/**`	`/docs`

Do not author in `hoox-landing-page/content/*` — that is a sync artifact.

Invariants & edge cases

1. **Python wins** language semantics. PyneTS does not invent TradingView® behaviour.
2. **AXIS ≠ engine.** AXIS never embeds a closed interpreter; engines are plugins.
3. **AXIS ≠ execution.** Drawing a strategy on AXIS does not call `trade-worker`.
4. **Alerts ≠ orders.** `alert()` webhooks are not `StrategyEvent` forwards unless you point them at the HOOX gateway yourself.
5. **Do not** `pip install pyne` or `pip install pynescript` for this project. Dist name is **`hoox-pyne`**.
6. **Do not** `npm install pynets` for this project. The published package is **`@hoox-sh/pynets`** from the standalone hoox-sh/pynets repo. Python Runtime remains the oracle.
7. **`pine-worker`** is a leftover TypeScript Cloudflare® experiment (hoox-sh/pine-worker). It is not a product evaluate host and has no pages on this site.

Worked examples

Pick a surface

You want	Use
Parse / lint / LSP on a laptop	`pip install "hoox-pyne[lsp]"`
Embed evaluate in TypeScript	`bun add @hoox-sh/pynets`
HTTP evaluate at the edge	Deploy **pyne-worker**
Upload then keep running (Flask/VPS/container)	`PYNE_RUNNER=1` — hosted runner
Chart + editor	AXIS PWA + Flask or Pyodide
Live CEX orders from strategy events	pyne-worker `TRADE_SERVICE` → HOOX
Chat → script	**pyne-agent-worker**

Failure modes

Mix-up	What goes wrong
Opening `pyne-worker/` or `pine-worker/` inside PYNE	Directories **do not exist** — clone the sister repos; TS library work is `@hoox-sh/pynets`
Expecting AXIS to flatten a position	No trade-worker hop
Pointing AXIS at pynets as an engine	No such engine. Use Flask / Pyodide / Worker proxy
`pyne run` vs `Runtime.run` vs `pynets run`	Different defaults — see modes

See also

- PYNE index
- PyneTS
- pyne-worker
- pyne-agent-worker
- Modes
- Evaluate contract
- AXIS

COMMERCIAL

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Commercial Licence" description: "AGPL escape hatch per organisation — closed-source use and SaaS without source disclosure. Licence sales opening soon."

COMMERCIAL LICENCE

Upcoming · per organisation · lifetime · sales open soon.

PYNE is AGPL-3.0-or-later. This licence is the alternative for closed use: pick **either** AGPL **or** commercial for the same code.

What it removes

AGPL requires	Commercial waives
Publish modifications	Keep them private
Offer Corresponding Source to SaaS users (§13)	No source offer to your users
Closed-source distribution forbidden	Ship inside closed products

Scope

- 1 legal entity, unlimited devs / products / servers.
- Lifetime for 0.x/1.x line (future 2.x may need new licence).
- Subsidiaries and clients need their own.
- No warranty, no support, no SLA, no API quota. Legal permission only.

Full text: `COMMERCIAL-LICENSE.txt`.

Buy

Commercial licensing is **upcoming** — not yet on sale. To register interest: sales@hoox.dev.

FAQ

Do I need it for internal use? No — internal AGPL use without distribution/hosting for third parties needs no commercial licence.

Do I need it for SaaS? Yes, if you host PYNE for third parties and don't want AGPL §13 source obligations.

Is hosted Pro API quota included? No. API call quotas (hobby/pro/team) are a separate product.

VAT? Merchant-of-record and EU VAT handling will be announced at launch.

Refund? 14-day EU digital-goods withdrawal terms will be published at launch.

COMPATIBILITY

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Compatibility guarantee" description: "Grouped map of what PYNE implements for Pine Script™ v5/v6 — working, partial, residual, and by-design out of scope. hoox-pyne 0.6.12."

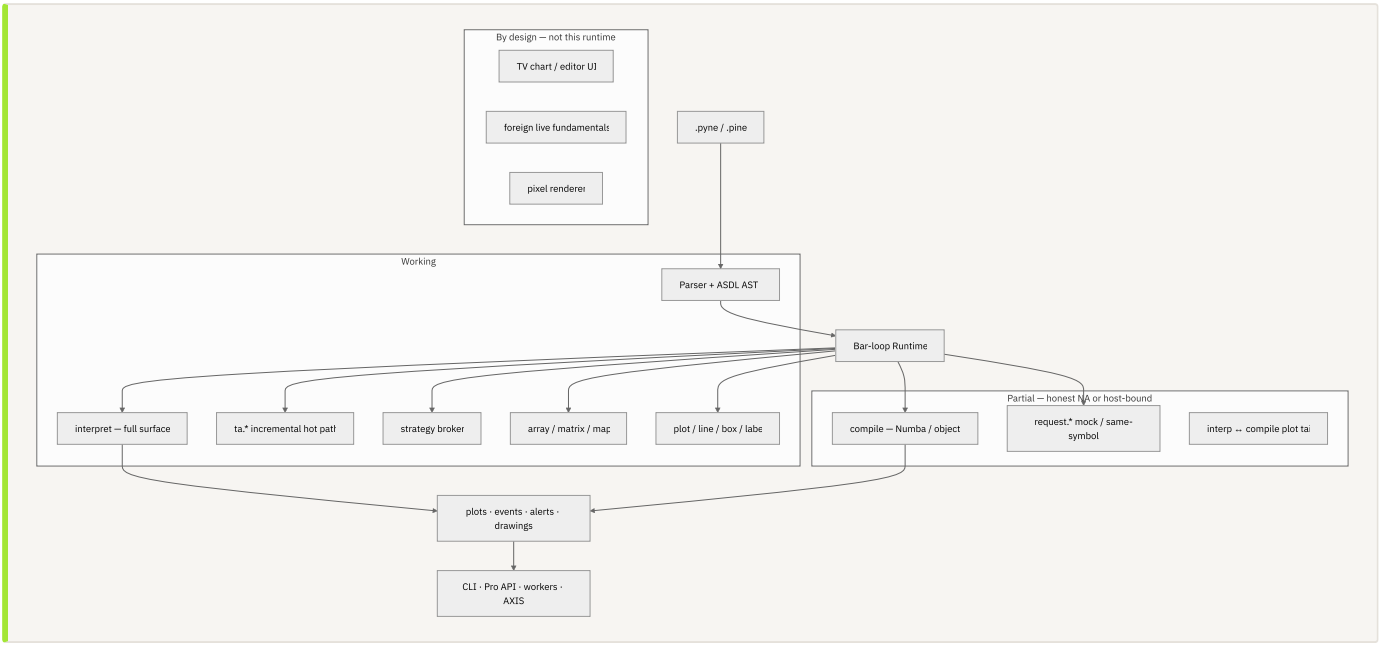
COMPATIBILITY GUARANTEE

Abstract

PYNE implements Pine Script™ **v5/v6 language core** as an inspectable pipeline: parse → AST → bar-loop evaluate. The map below is the product-facing compatibility surface for `hoox-pyne` **0.6.12**. It is **not** TradingView® platform identity (chart host, proprietary data, editor-only UI) and **not** bit-identical results vs the hosted platform on every script or bar.

Name-level dispatch lives in Pine v6 surface (`docs/pine_v6_full_surface_inventory.md`). Gaps: Missing features.

Conceptual model



Legend

Mark	Meaning
Working	Implemented, usable, covered by first-party tests
Partial	Callable but mock, host-bound, or incomplete vs hosted Pine
Residual	Known open hole on an otherwise landed surface
Out of scope	Intentionally not a TV platform clone

Dispatch snapshot (2026-10-08): **941** callables, **10** partial-heuristic, **0 missing** vs the public TV v6 function list (434 symbols). A registered name is not the same as hosted-platform semantics.

```
Language, series, TA hot path, collections, strategy broker, drawings, alerts, libraries, interpret

`request.*` feeds (honest `na`), compile eligibility, compile alerts, PyneTS vs Python oracle.

Plot MISMATCH corpus tail (P1p), `PYNE_SERIES_RING` default off.

Pixel chart, editor UX, foreign live data, bit-identical TV bars.
```

Interface surface

Corpus (set01–04 · 2477 scripts · 2026-08-09)

Open-source sets, **not shipped** in git. Not a TradingView® platform score.

Suite	Rate	Notes
Parse + unparse	**99.96%** (2476/2477)	Residual: one intentional invalid line-wrap demo
Runtime interpret (50 bars)	**100%** excl. EXPECTED_FAIL	2466 OK + 11 listed demos
set01 Runtime	**249/249**	Interpret (and prior compile sweep)

EXPECTED_FAIL are path-listed demos (`runtime.error` library guards, lower-TF security, pathological loops) — not silent suppression.

Grouped map

```
| Area | Status | Notes |
| --- | --- | --- |
| ANTLR4 grammar v5/v6 | Working | Multiline strings, soft keywords, bitwise, typed UDF returns (`F
| Parse → unparse | Working | Structural fidelity; whitespace may differ |
| `var` / `varip` / `:=` | Working | Execution-site init; start-of-bar carry |
| Functions, methods, type annotations | Working | |
| Control flow, `switch`, `once`, enums | Working | Strict bool; `na` is false in conditions. `once
| UDTs + `.new` | Working | Field defaults on compile |
| Libraries `export` / `import ns/Name/ver` | Working | In-process `register_library_source`; `Runt
| String interpolation, tuple unpack, method chaining | Working | |
| Series `[]` history | Working | OOB / negative → `na` (not wrap) |
| Line wrapping | Working | Any indent inside `(…)` and `[…]`; `table.new` is columns-then-rows (Pi
| `timeframe.change` | Working | Intraday fixed-width buckets; D/W/M on the exchange calendar (`sym



| Area | Status | Notes |
| --- | --- | --- |
| `pynescript.runtime.Runtime` | Working | Package SoT; `backend.runtime` re-exports |
| Modes `interpret` / `compile` / `auto` | Working | Defaults differ by surface — [modes](/pyne/doc
| Series caps | Working | `PYNE_SERIES_CAP` default **on** |
| Incremental `ta.*` | Working | Default **on**; `PYNE_TA_INCREMENTAL=0` full recompute |
| Derived OHLCV skip | Working | Unused `hl2`/`hlc3`/`ohlc4`/`tr` not written; `ta.vwap` still upda
| `timeout_seconds` | Working | Library / edge / Flask `/run` + `/run/batch` (omit = no budget) |
| Ring buffer | Residual | `PYNE_SERIES_RING` default **off** |



| Area | Status | Notes |
| --- | --- | --- |
| MAs | Working | `sma`/`ema`/`rma`/`wma`/`hma`/`vwma`/`kama`/`dema`/`tema`/`alma` incremental |
| Oscillators | Working | `rsi`/`macd`/`stoch`/`cci`/`cmo`/`tsi`/`roc`/`wpr`; incremental `aroon`/`
| Volatility / structure | Working | `atr` (Wilder `rma(tr)`), `bb`, `kc`, `stdev`, `highest`/`lowe
| Volume | Working | Incremental `obv`/`wad`/`wvad`/`cmf`/`klinger`/`mfi`/`vwap`/`nvi`/`pvi` |
| Supertrend | Working | Locked mid±factor·ATR (inc ≡ compile ≡ Numba). TV band ratchet is out of s



| Area | Status | Notes |
| --- | --- | --- |
| `array.*` | Working | Negative indices; UDT `sort_field` + `binary_search*` |
| `matrix.*` | Working | LA (`det`/`inv`/`eigen`…), predicates, `sort_field` |
| `map.*` | Working | |



| Area | Status | Notes |
| --- | --- | --- |
| `entry` / `close` / `cancel` / events | Working | `StrategyEvent` stream |
| `exit` stop/limit + OCA | Working | OHLC path, not tick path |
| `profit` / `loss` | Working | Ticks × mintick from entry avg |
| `from_entry` / `qty_percent` | Working | Interpret + compile |
| Trail | Working | `trail_offset` / `trail_points` / `trail_price` on OHLC high/low (dual-host). T
| Risk cascade | Working | `allow_entry_in`, max size/drawdown/cons-loss-days/intraday |
| Open/closed trade fields + MAE/MFE | Working | Extremes from bar high/low |
| Pending-fill average (pyramiding ≤ 0) | Working | F2 |
| Licensed broker / exchange fills | Out of scope | In-process model only |
```

Area	Status	Notes
`plot` / `plotshape` / `plotchar` / `plotarrow` / `hline` / `bgcolor` / `barcolor` / `fill` / `pl
`line` / `box` / `label` / `table` / `polyline` / `linefill`	Working	Coordinates snapshotted
Drawing export parity	Working	Both hosts emit cell style/width/height/tooltip, `force_overlay
`max_*_count` GC	Working	Interpret registry
`force_overlay`	Working	Export payload
Pixel paint	Out of scope	AXIS / clients consume series + drawings
Compile `hline`/`fill`/`bgcolor`/`plotshape` keys	Working	First-party key sets match interpre

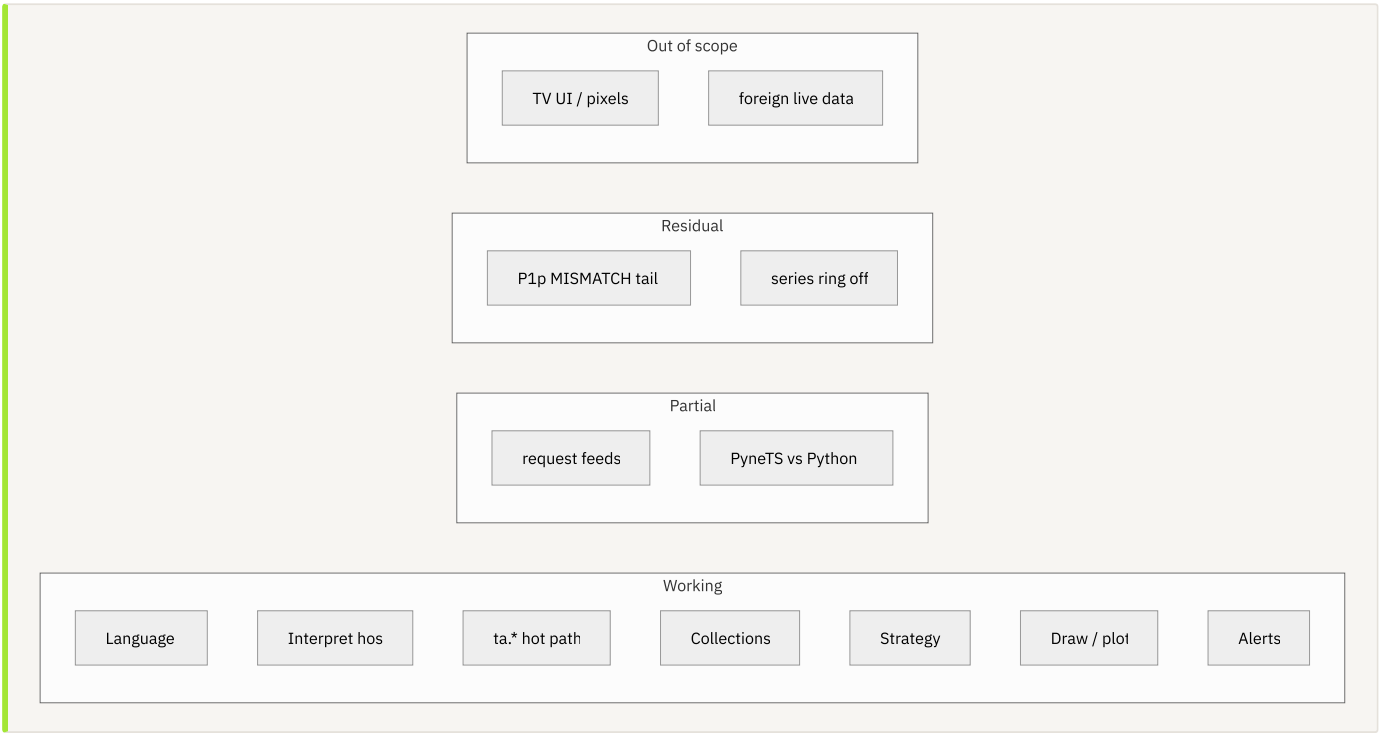
Area	Status	Notes
Same-symbol simple OHLCV / HTF	Working	Allowlisted `ta.sma`/`ema`/`rsi`/`atr`/`wma`/`rma` on
Foreign / complex `request.security`	Partial	Resolves to **`na`** (no invented chart-close)
`request.footprint` / financial / economic	Partial	Mock / feed-scaled; not a live vendor
`input.*` including `active` / `enum`	Working	Values + metadata
Bid/ask omitted	Partial	`na` when host does not supply
Real multi-symbol market data (B1)	Out of scope	Needs a host adapter

Area	Status	Notes
`alert()` / `alertcondition()`	Working	Frequency rules; `/run` export
L2 webhooks	Working	`webhook_url` / `ALERT_WEBHOOK_URL`
Compile alerts	Partial	Interpret is the alert oracle
Numba + object-mode compile	Working	`mode=auto` falls back; `import` / `request.*` / `inputs=
Warm compile / disk IR	Working	Prewarm CLI/API; cache meta v18
Interp ↔ compile plot parity	Working	Harness + first-party goldens + official TV builtins **0
CLI / LSP / Pro API / Docker	Working	`pyne` / `pyne-lsp`; Flask `/run`
Flask `/run` `libraries`	Working	Max 32; also `/run/batch`
Flask `timeout_seconds`	Working	Optional on `/run` and `/run/batch`; omit / `≤ 0` = no timeou
`timeframe.change`	Working	Intraday fixed-width; D/W/M exchange calendar. Interpret + compile
`POST /optimize` / `pyne optimize`	Working	Strategy `input.*` search — not a Pine builtin

Surface	Status	Notes
Python Runtime	Working	Language oracle
PyneTS `@hoox-sh/pynets` 0.5.0	Partial	Interpret + JS compile on npm; Python Runtime is the c
pyne-worker	Working	Thin Python CF host; vendors engine (may lag PyPI). Docs: [pyne-worker](/
pyne-agent-worker	Working	NL authoring; optional `/run` validate. Docs: [agent](/pyne/docs/ag

Residual and out of scope (compact)

Item	Class
Interpret ↔ compile value `MISMATCH` (optional set0x community corpus)	Residual (P1p official builtins closed)
`PYNE_SERIES_RING` default-on	Residual (flagged off)
Pixel chart / editor word-wrap / TV UI	Out of scope
Foreign live fundamentals without a feed	Out of scope
TV Supertrend band ratchet	Out of scope
Tick-path trail / broker	Out of scope
Bit-identical every recursive smoother vs TV	Out of scope
Parallel bars / whole-script vectorization	Out of scope



Dual-host plot parity

Same script + OHLCV under `mode="interpret"` and `mode="compile"`; series compared with nan-aware allclose (`rtol=1e-5`, `atol=1e-6`).

Artifact	Role
`scripts/compare_interp_compile.py`	Corpus harness → `.cache/interp_compile_parity.json`
`tests/test_interp_compile_parity.py`	Always-on smoke
`tests/test_first_party_ta_goldens.py`	ATR / Supertrend / Keltner dual-host

Known **contract** differences (not silent value bugs):

1. Foreign / complex `request.*` → `na` on both hosts when no feed.
2. First-party `hline` / `fill` / `bgcolor` / `plotshape` keys match; harness `--ignore-hline-keys` / `--ignore-fill-keys` remain optional for leftover corpus noise.

3. Pivot-starved Auto Fib → matching `runtime.error` (`both_error_same`), not invented pivots.

Internals

Artifact	Role
`docs/compatibility_guarantee.md`	Historical long-form; this page is the map
`COMPATIBILITY.md`	Repo-root distillation of this map
`docs/pine_v6_full_surface_inventory.md`	Name-level dispatch tables
`docs/known_divergences.md`	Semantic deltas vs public Pine docs
`tests/fixtures/parity/`	First-party strategy fixtures

Invariants & edge cases

1. **Parse success ≠ evaluate success.** Round-trip can still depend on `request.*` data.
2. **Dispatch success ≠ hosted semantics.** 0 missing vs the public function list does not mean TV-identical bars.
3. **Interpret is the oracle.** Compile matches where eligible; `mode=auto` falls back.
4. **Plots are not a renderer.** Registry + series export; pixels are AXIS / clients.
5. **Floating-point is statistical** for some recursive smoothers — Numerical validation.
6. **Libraries** need in-process registration; there is no TV CDN.

Worked examples

Round-trip

```
from pynescript.ast.helper import parse, unparse

src = open("strategy.pine").read()
assert unparse(parse(src))  # structural; whitespace may differ
```

Interpret ↔ compile

```
python scripts/compare_interp_compile.py --bars 1000 --limit 50
python scripts/compare_interp_compile.py --ignore-hline-keys --ignore-fill-keys
pytest tests/test_interp_compile_parity.py -q
```

Failure modes

Observation	Read as
Parse error on a new TV release-notes feature	Surface gap — Missing features
Numerical drift on a custom smoother	Bar-mode vs list-mode; `na`; known divergences
Interpret vs compile series mismatch	Foreign `request.*`, UDF last-assign `na`, leftover corpus keys
Strategy equity ≠ TV	Commission / slippage / OHLC vs tick path
`import` unresolved	Library not in `libraries=` / registry
`/run` 400 `UNKNOWN_FIELDS`	Extra key not in `RUN_SCHEMA`

See also

- Missing features
- Pine v6 surface inventory
- Numerical validation
- Runtime modes
- Roadmap

MISSING FEATURES

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Missing features" description: "Known gaps and closed Pine Script™ v6 items — what is implemented, what is partial by design, and what is out of scope. hoox-pyne 0.6.12."

MISSING FEATURES

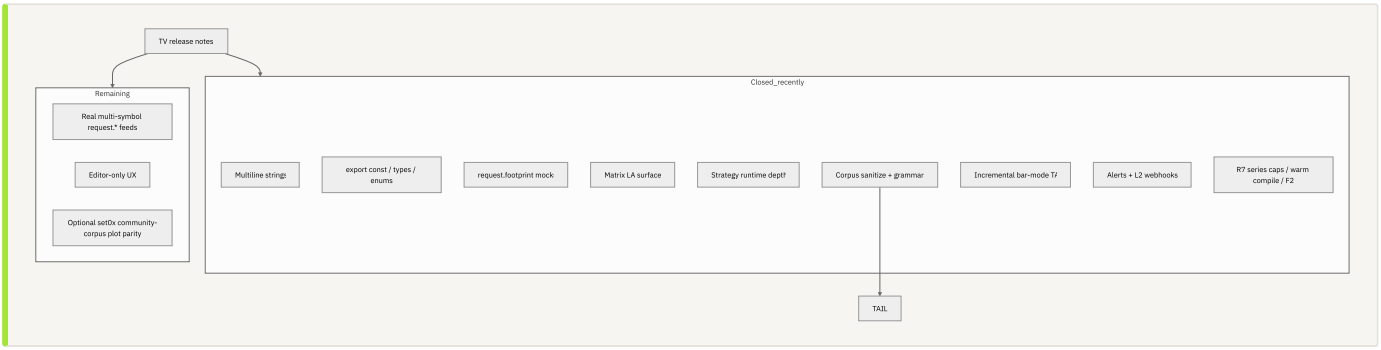
Abstract

"Missing" in PYNE is a **moving frontier**. Core v6 support is ~99%+ for core language, with post-launch TV items closed (multiline strings, library `export const`, footprint mocks, risk enforcement, matrix LA, compile-mode strategy, drawing GC, bottom-panel table fields, square-bracket wrapping) plus open-source corpus parse/Runtime hardening, incremental bar-mode TA, alert engine + L2 webhooks, series caps, warm-compile, pending-fill averaging, `timeframe.change` (intraday buckets + D/W/M exchange calendar), package Runtime SoT

(`pynescript.runtime` ; `backend.runtime` is a shim), compile object-mode residual recovery (UDF locals, nopython `None` /unicode, matrix handles, UDT returns, `x = switch` , drawing copy, `int(na)` , `round()` extras, import stubs, color type dispatch, nested UDT methods), type-aware intraday risk enforcement on both brokers, intrabar `var` rollback on realtime ticks, `request.security` barmerge on HTF resample paths, linter legacy-rule retirements, and drawing export parity (table/box/label/polyline style fields on both hosts). 0.6.10-0.6.12: engine-owned live plane (WS `/ws/live`), refcount-pooled shared sessions (F-29), opt-in delta envelopes, gevent WebSocket worker (F-30), CCXT provider caching + tick-walk sharing, live session persistence (SQLite journal), production profile + `PYNE_DATA_DIR` , Docker prod on VPS, Redis unpublished + opt-in auth, CORS for AXIS authenticated requests. Remaining gaps cluster into optional set0x community-corpus plot parity (P1p official builtins closed), by-design platform differences, and editor-only features.

Authoritative narrative: `docs/missing_features.md` . Canonical IDs: `docs/ROADMAP.md` .

Conceptual model



Interface surface

Recently completed (do not re-open as "missing")

Summarized — full prose in source doc:

Area	Notes
Multiline `"""` / `'''`	Lexer + LexerBase + unparser
Library `export const` + types/enums	Parse/AST/runtime registry
UDT `sort_field` on array/matrix sort	Implemented
UDT `sort_field` on `array.binary_search*`	Implemented (interpret + compile)
`once` conditional structure	Implemented — parse/unparse, interpret (confirmed-bar latch + realtime re-fire), compile historical flags. Statement-only.
`input.*.active`	Metadata-driven
Dynamic `request.*`	Shared resolvers; mock/feed scaling
Enums runtime + input.enum	Type kind + LSP partial
Strategy open/closed trades, risk, OCA, commission	Golden tests
Drawing `*.all`, plot registry, **`max_*_count` GC**	Effects recorded; GC on package + Pro + AXIS
Numba compile + object mode + **warm-compile (H2)**	Disk IR cache, prewarm API/CLI, deploy defaults
Compile object-mode residuals	UDF locals / free-series, nopython `None` / `timestamp`, matrix handles, UDT returns, `x = switch`, drawing copy, `pine_int(na)`, sanitize chrome, `round()` extras, import stubs as `na`, color type dispatch, nested UDT methods
Official matrix LA and predicates	det/inv/eigen/...
0 missing vs public TV v6 function reference list (434 symbols)	
Runtime `_pine_defs_locked` + append-only `current_series`	Host hygiene (backend + pyne-worker)
Incremental `ta.*` (MAs, oscillators, `bb` / `kama` / `stochrsi`, volume `obv` / `wad` / `cmf` / `klinger` / `nvi` / `pvi`)	Bar mode; `PYNE_TA_INCREMENTAL=0` to disable
Series caps (T1)	`PYNE_SERIES_CAP` default ON + goldens
Pending-fill averaging (F2)	Pyramiding ≤ 0; interpret + compile broker goldens
Alert engine + **L2 webhooks (L2)**	Dual-host `/run` export, `ALERT_WEBHOOK_URL` / `webhook_url`
`request.security` foreign-na policy	Foreign/complex → `na` both backends; same-symbol OHLCV passthrough
Plot parity harness (P1p)	`scripts/compare_interp_compile.py`, first-party goldens + official TV builtins **0 MISMATCH**
`fill()` export for AXIS	series + `plot_meta` / compile `__drawings`
Square-bracket wrapping	Any-indent line wrapping inside `[…]`; `table.new` columns-then-rows
Drawing export parity	`table.cell` style storage + reference positional order; cell style/width/height/tooltip, `force_overlay`, `merged_cells` both hosts; box/label text extras; polyline curve/fill; `text.align_*` constants

Still incomplete / by design

Item	Status posture
Real (non-mock) multi-symbol `request.*` market data (**B1**)	By design for library core; feeds optional. **Foreign-na policy landed** (no invent).
`request.security` foreign data	**Not filled with real series** — emits `na` for foreign/complex (honest). Same-symbol simple OHLCV may lower.
Auto Fib / pivot-heavy scripts	Need real pivot structure (or registered ZigZag lib). Flat synthetic bars → intentional `runtime.error` (insufficient pivots); both hosts should agree (`both_error_same` in parity harness).
Pixel chart host	External (AXIS / clients)
Some unlimited-history / trim edge cases	High-level support; exotic TV behaviors may differ
Editor word-wrap / UI-only release notes	N/A
PyneTS full builtin parity (**L3**)	interpret + JS compile landed in `@hoox-sh/pynets` 0.5.0; Python remains oracle. Open rows: PyneTS missing features
Bit-identical every recursive smoother vs TV	Numerical bounds, not bits
Dual Runtime host copies (**H1**)	**Largely done** — package `pynescript.runtime` SoT + `backend.*` shims + pyne-worker thin wrap. Residual = worker-only extras (logs/profile, CF first-plot), not a forked bar loop
Warm-compile product path (**H2**)	SLOs / prewarm / deploy IR cache defaults
Runtime residual (**C1**)	set01-04 Runtime interpret **100%** excl. EXPECTED_FAIL; third-party corpus not shipped
Series cap (**T1**)	`PYNE_SERIES_CAP` default ON
Residual TA inc (**T2**)	R7 bb/kama/cmo/stochrsi + wma/hma/linreg; volume `obv` / `wad` / `cmf` / `klinger` ; `nvi` / `pvi` ; `aroon` / `dpo` / `donchian` / `kst`
Pending-fill (**F2**)	R7 interpret + compile goldens
Plot parity residual (**P1p**)	Harness + first-party goldens + official TV builtins **0 MISMATCH**; optional set0x community corpus unmeasured
Compile leftover library stubs	Some `float(None)` / UDF arity on missing `import` libraries; INV expected-fail fixtures stay fail
TV Wilder RMA-ATR / supertrend (**F1**)	interpret ATR is `ta.rma(ta.tr)` ; Supertrend mid±factor·ATR locked. TV ratchet is out of scope
Alert webhooks (**L2**)	Pro API + pyne-worker; see Alerts

Post-v6 launch themes (historical gap list)

v6 launch items (dynamic requests, strict bool, enums, polylines, `log.*` , negative indices, truediv, ...) are largely supported or intentionally stubbed where platform-bound.

2025-2026 monthly TV updates introduced footprint types, `active` inputs, multiline strings, `export const` , UDT sort fields — tracked and largely closed.

Corpus + performance snapshot

Open-source set01-04 (**2477** scripts; measured locally, **not shipped** in-repo):

Metric	Value
Parse + unparse	**99.96%** (2476 / 2477) — residual: 1 intentional invalid line-wrap docs demo
Runtime interpret	**100%** excl. EXPECTED_FAIL — 2466 OK + **11** intentional demos
set01 Runtime	**249 / 249 (100%)**
EXPECTED_FAIL class	library `runtime.error` demos, lower-TF security guards, pathological loops, truncated mid-call scrapes
`ta_sma` bar loop (≈3.2k bars)	**~9.5×** vs full recompute
`ta_atr` bar loop	**~10×**
macd+atr+rsi+sma combo	**~8.5×**

Not TradingView® platform parity. OK% excludes only path-listed intentional demos (never soft-suppress unknown failures).

Golden: `tests/test_ta_incremental.py`, `tests/test_series_cap.py`.

Internals

Path	Role
`docs/missing_features.md`	Living gap analysis
`tests/test_v6_features.py`	v6 feature coverage
`tests/test_pine_surface_gaps.py`	Surface gap probes
`tests/test_ta_incremental.py`	Incremental TA ≡ full recompute
`tests/test_series_cap.py`	Series cap goldens (T1)
`scripts/compare_interp_compile.py`	Plot parity harness (P1p)
`docs/pine_v6_full_surface_inventory.md`	Full name-level inventory
`resource/*.g4` + builder	Grammar-level feature work
`.../technical_submodules/core.py`	`_sma_inc_update`, `_macd_inc_update`, `_atr_inc_update`, …

Invariants & edge cases

1. **Closing a syntax gap requires grammar + builder + unparser + tests**, not only an evaluator stub.
2. **Mock data can green tests while production feeds differ** — declare data source explicitly.
3. **Compile does not fill foreign `request.security` data** — multi-asset plots stay `na` on compile unless same-symbol simple OHLCV; do not treat invent-as-close as a fix.
4. **Auto Fib needs pivots** — without swing structure (or ZigZag library registration), expect structured insufficient-pivot errors, not fib lines.
5. **"0 missing vs reference list" ≠ identical semantics for every edge case.**

6. **Perf changes must keep bar-by-bar semantics** — no whole-script vectorization; prefer golden tests vs current oracle.
7. Update this annex when `missing_features.md` changes major status lines.

Worked examples

Detect multiline string support

```
from pynescript.ast.helper import parse, unparse
src = '''//@version=6
indicator("m")
s = """a
b"""
plot(1)
'''
tree = parse(src)
assert '"""' in unparse(tree) or "'''" in unparse(tree)
```

Footprint mock call

Prefer tests in `tests/test_v6_features.py` as executable specification for `request.footprint` methods.

Incremental TA parity smoke

```
.venv/bin/python -m pytest tests/test_ta_incremental.py -q
# Disable hot path if needed:
PYNE_TA_INCREMENTAL=0 python -c "from pynescript.runtime import Runtime; ..."
```

Failure modes

Symptom	Likely gap class
Lexer error on triple quotes	Stale generated lexer (should be fixed on main)
Import member missing	Library export registration
Empty footprint rows	Mock seed / data_feed not configured
Risk not blocking entry	Old runtime path; ensure risk_kwargs applied
Runtime TIMEOUT on method-heavy scripts	Missing `_pine_defs_locked` on host
Corpus PARSE_FAIL mid-block	Truncated scrape — not a grammar hole

See also

- Pine v6 surface
- Roadmap
- Compatibility
- Technical analysis (`ta.*`)
- Alerts

- Interpret ↔ compile parity

PINE V6 SURFACE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Pine v6 surface inventory" description: "Summary of the full Pine Script™ v6 surface inventory — dispatch counts, namespaces, status schema — with pointer to the large source document. hoox-pyne 0.6.12."

PINE V6 SURFACE INVENTORY

Abstract

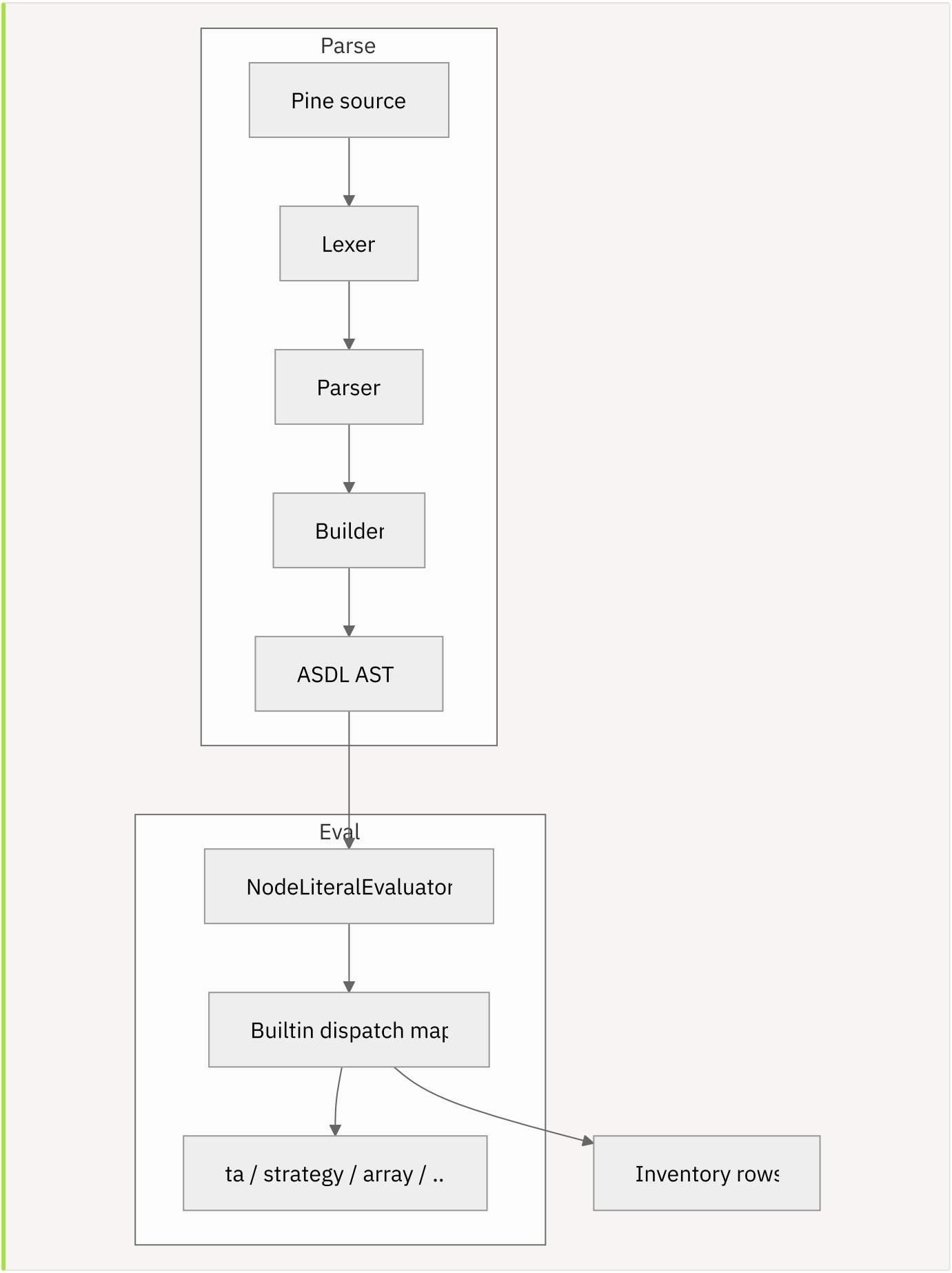
The **full surface inventory** enumerates every registered evaluator builtin, major series/variable, language construct, and known gap against Pine Script™ v6. It is deliberately large (tens of thousands of tokens when fully expanded). **This page is the navigable summary**; the exhaustive tables live in-repo at:

`docs/pine_v6_full_surface_inventory.md`

Generated snapshot referenced here: **2026-10-08**, from live `NodeLiteralEvaluator` dispatch, `builtin_metadata.json`, and design docs.

Do **not** paste the entire inventory into product docs — regenerate and link.

Conceptual model



Interface surface

Status schema

Status	Definition
Implemented	Handler/parser path exists and is usable
Partial	Mock/stub, incomplete semantics, or metadata-only
Missing	Expected on TV v6; not resolving
By design	Intentionally not full TV platform
N/A	Editor/UI-only

Each inventory row also carries: **Name**, **Namespace**, **Kind**, **Metadata** (LSP JSON present?), **Source** (e.g. `dispatch`), **Notes**.

Summary counts (2026-10-08)

Metric	Count
Dispatch builtins (callable)	**941**
Dispatch partial-heuristic (stub/mock docstring)	10
Dispatch namespaces	255
Official TV v6 function reference list	434 symbols — **0 missing** in dispatch

Top namespaces by dispatch keys

Namespace	Count
`ta`	165
`strategy`	95
`matrix`	74
`array`	68
`box`	32
`label`	29
`table`	27
`color`	26
`math`	25
`line`	24
`str`	19
`plot`	15
`timeframe`	15
`input`	14
`polyline`	14
`map`	11
`request`	11
`footprint`	9
`ticker`	9
(remaining namespaces)	smaller

Kinds covered

`function` · `series/var` · `constant` · `declaration` · `control` · `operator` · `type system` · `literal` · `semantics` · `export` · `import`

Internals

Path	Role
`docs/pine_v6_full_surface_inventory.md`	Full tables + architecture diagrams
`scripts/regenerate_inventory_summary.py`	Regeneration helper
`scripts/generate_builtin_metadata.py`	LSP metadata from live surface
Evaluator `_build_builtin_map()`	Source of dispatch truth

How to use the full file

1. Open `docs/pine_v6_full_surface_inventory.md` in the repo or Sphinx site mirror.

2. Search by namespace (`## ta` , `strategy` , ...).
3. Trust **dispatch** source rows for "is there a callable?"; read **Notes** for mock/feed caveats.
4. After large builtin PRs, regenerate summary counts before quoting them publicly.

Invariants & edge cases

1. **Inventory size is a feature** — summarization must not drop partial rows when claiming coverage.
2. **Metadata presence ≠ semantic completeness.**
3. **Partial heuristic count (10)** is docstring-based — real partials may be higher.
4. **Language constructs** (control/export) may be parser-complete without every runtime edge.

Worked examples

Quote coverage honestly

> "Against the public Pine v6 function reference (434 symbols), pynescript registered **0 missing** dispatch entries as of 2026-10-08; 941 total callables across 255 dispatch namespaces. See `docs/pine_v6_full_surface_inventory.md` ."

Programmatic check (sketch)

```
# regenerate after evaluator changes
python scripts/regenerate_inventory_summary.py
```

Failure modes

Mistake	Consequence
Pasting 134KB inventory into Mintlify page	Unreadable docs; broken UX
Citing counts without date	Stale marketing
Equating dispatch hit with TV gold	False precision claims

See also

- Missing features
- Compatibility
- Runtime builtins

NUMERICAL VALIDATION

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Numerical validation" description: "Methodology and error bounds for PYNE technical and math builtins versus reference Pine Script™ implementations."

NUMERICAL VALIDATION

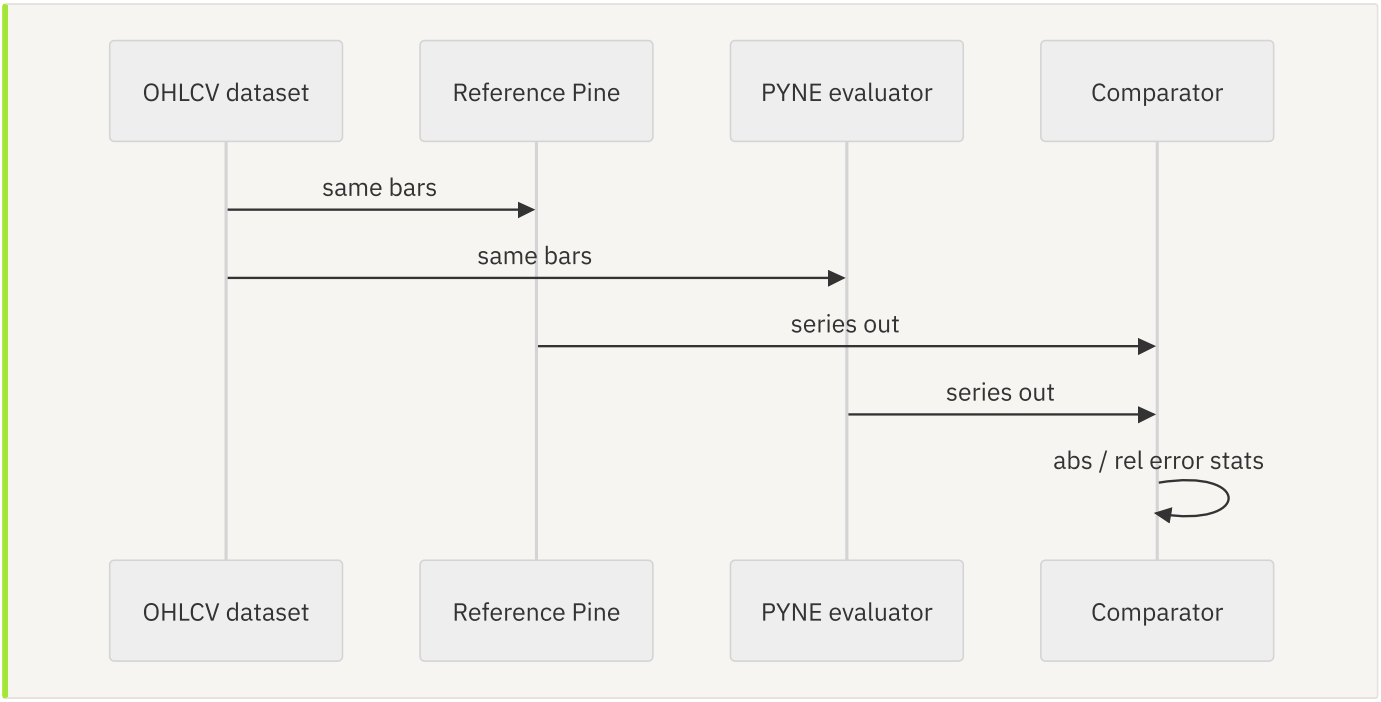
Abstract

PYNE's technical and mathematical builtins are validated for **numerical agreement** with reference Pine Script™ implementations under IEEE 754 realities. The long-form report (`docs/numerical_validation_report.md` , v1.0, Nov 2025) documents methodology, per-family error tables, and edge cases.

Headline (from that report): max observed relative error **0.0022%** (ADX, extreme volatility); average error **< 0.0005%**; deterministic ops (SMA, OBV, discrete math) often **exact**.

Treat percentages as **report-era measurements**, not eternal SLAs — re-validate after changing smoothers or bar-mode semantics.

Conceptual model



Interface surface

Acceptable thresholds (report)

Band	Relative error	Verdict
Excellent	< 0.001%	Pass
Good	< 0.01%	Pass
Acceptable	< 0.1%	Review
Unacceptable	≥ 0.1%	Fail

Metric definitions

- **Absolute error:** `|ref − pyne|`
- **Relative error:** `|ref − pyne| / |ref| × 100%` (guard zero refs)
- **Max / mean / std** over bars and scenarios

Category highlights

Family	Notes
Moving averages	SMA exact; EMA/WMA/... tiny FP error
Oscillators	RSI/stoch slightly higher smoothing error still ≪ 0.01%
Trend	ADX highest in report (0.0022% max)
Volume	OBV exact cumulative; MFI inherits typical-price FP
Stats	percentrank exact; variance/stdev excellent
Math	Discrete exact; transcendentals ~1e-15 class

Dual-host formula alignment (interpret ↔ compile)

Where fixed, these kernels **share one formula contract** across interpret (evaluator / incremental) and compile (`numba_builtins`) so plot series match bit-identical or maxdiff ≈ 0 on dual-run goldens:

Builtin	Shared contract (summary)
`ta.rsi`	Wilder: SMA seed of first `period` deltas, then RMA gain/loss; first valid at bar `period`
`ta.roc`	Standard lookback % change; `na` until lookback (no early `0.0`)
`ta.wma`	Full non-`na` window required; no partial-window reweight (nested ROC/WMA safe)
`ta.cum`	Running sum; NaNs as 0; user series (`ad = ta.cum(...)`) must not alias builtin A/D
`ta.highestbars` / `lowestbars`	Negative bars-back offset; short/all-NaN → `-1`; oldest extreme on ties

Goldens: `tests/test_compiler_numba.py` (`TestInterpCompilePlotParityFixes`) and `scripts/compare_interp_compile.py`. Residual dual-host drift (if any) tends to cluster in **nested EMA seed families** (DEMA/TEMA SMA-seed vs first-value) rather than these aligned kernels — see Compatibility.

Error distribution (report aggregate)

Error range	Share (approx.)
Exact 0	~38%
< 0.0001%	~46%
0.0001–0.001%	~14%
0.001–0.01%	~2%
≥ 0.1%	0%

Internals

Path	Role
`docs/numerical_validation_report.md`	Full tables + bias analysis
`tests/test_ta_indicators_*.py`	Executable TA tests
`tests/test_indicators.py` / builtins tests	Additional numeric guards
`tests/test_compiler_numba.py`	Interpret ↔ compile formula goldens (RSI/ROC/WMA/...)
`scripts/compare_interp_compile.py`	Corpus plot-series parity harness
Bar-mode vs list-mode (`_pine_bar_mode`)	Scalar-per-bar vs full series — compare like with like

Methodology (report)

1. Generate synthetic 1k-bar OHLCV across regimes (trend, range, vol, gaps)
2. Validate with real histories (equities, crypto, FX samples)
3. Export reference outputs; run PYNE; element-wise compare
4. Search systematic bias; inspect tails

Invariants & edge cases

1. **na propagation** must match Pine truthiness — numerical compare should mask `na` pairs.
2. **Bar-mode scalars** vs full-series list mode can look like “bugs” if misaligned.
3. **Seeded mocks** (`request.seed`) stabilize stochastic feeds for regression.
4. **Recursive smoothers** accumulate FP differently across languages; bounds > bits.
5. **Extreme prices** (near-zero, huge) tested; still within excellent band in report.

Worked examples

Relative error helper

```
def rel_err(a: float, b: float) -> float:
    if a == 0 and b == 0:
        return 0.0
    denom = abs(a) if a != 0 else abs(b)
    return abs(a - b) / denom * 100.0
```

Run TA tests

```
pytest tests/test_ta_indicators_1.py tests/test_ta_indicators_2.py -q
```

Failure modes

Symptom	Investigation
Sudden ADX drift	Check Wilder smoothing / seed bars
SMA not exact	Off-by-one window or float input casts
Good unit tests, bad TV export compare	Timezone/session alignment of bars
Only bar 0 differs	Warm-up / `na` initialization

See also

- Compatibility
- Runtime technical builtins

ROADMAP

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Roadmap" description: "PYNE forward plan — dual-host residual, runtime residual, plot parity harness; drawing export parity landed. hoox-pyne 0.6.12."

ROADMAP

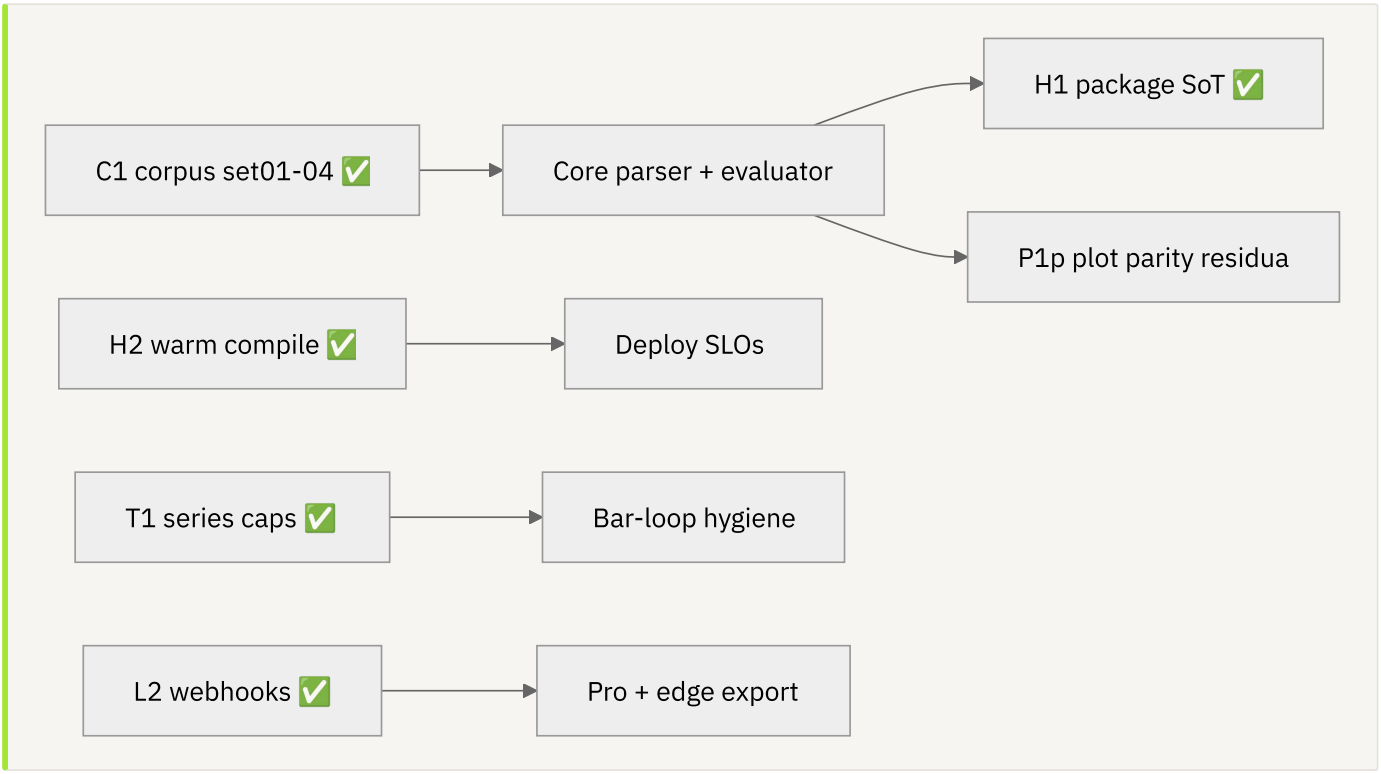
Abstract

The roadmap is a **living prioritization document**, not a contract. Source: `docs/ROADMAP.md`. Core v6 language/builtins are essentially closed. Product **warm-compile (H2)**, **series caps (T1)**, **pending-fill (F2)**, **L2 webhooks**, **corpus Runtime residual (C1)**, **package Runtime SoT (H1 largely done)** and **drawing export parity** have **landed** (set01–04 parse **99.96%**, Runtime interpret **100%** excl. intentional demos). **0.6.10–0.6.12:** engine-owned live plane (WS `/ws/live`), refcount-pooled shared sessions (F-29), opt-in delta envelopes, gevent WebSocket worker (F-30), CCXT provider caching, live session persistence (SQLite journal), production profile +

`PYNE_DATA_DIR`, Docker prod on VPS, Redis unpublished + opt-in auth. What remains: **H1 worker-only extras**. **P1p** official TV builtins are **0 MISMATCH** (optional set0x community corpus unmeasured). Incremental TA and Supertrend goldens have landed.

Conceptual model

```
P0 docs honesty → P1 dual-host H1 ✅ (package SoT + shims + worker thin wrap)
                → P1 plot parity residual (P1p)
                → H2 warm compile ✅ · T1 series caps ✅ · F2 pending-fill ✅ · C1 corpus ✅
                              ↘ T2 nvi/pvi ✅ · F1 Supertrend goldens ✅
                                  → P6 long-horizon (L1 / L3; L2 ✅)
```



Interface surface

Status snapshot

Component	Status
Parser ANTLR4	v5/v6 + multiline / export const
Evaluator	incl. full var/varip, ReAssign
Builtins / TA / collections	broad (0 missing vs public TV ref list)
Strategy events + broker (OCA, commission, risk)	
Numba + object-mode compile	MVP+ (disk IR cache, `mode=auto`, `time_arr`, cache recovery; UDT/switch/matrix/drawing/round/color residuals)
Pro API + auth + Docker	(`mode` default `auto`; targets api / api-dev / lsp / **cli**)
PyPI **`hoox-pyne`**	live **0.6.12** (pypi.org/project/hoox-pyne); CLIs `pyne` / `pyne-lsp`
CLI surface (`check` / `format` / `compile` / `run` / `prewarm` /...)	+ Nuitka binaries + CLI Docker image
Alert engine + L2 webhooks	dual-host export + outbound webhooks
Drawing `max_*_count` GC	package + Pro API + AXIS Pyodide
Drawing export parity (table/box/label/polyline)	cell style, `force_overlay`, `merged_cells`, text extras, curve/fill, `text.align_*`; both hosts identical
Package Runtime SoT (`pynescript.runtime`)	Pro API + worker thin wrap
Strategy exits (`from_entry`, `qty_percent`, trail, risk cascade)	
`request.security` HTF simple TA + foreign-na	same-symbol OHLCV/HTF allowlist; foreign → `na`
Free-tier Pro API guards	bar/script caps, rate, concurrency, mock-only free data
Dual-host TA goldens (ATR / Supertrend / Keltner)	first-party + `test_ta_incremental` CI
Interpret dispatch + unused derived skip + volume inc	interpret dispatch inlining, unused derived OHLCV skip, incremental volume TA
Series caps (`PYNE_SERIES_CAP`)	T1
Warm-compile product path	H2 (SLOs, prewarm, deploy IR cache)
Pending-fill averaging (pyramiding ≤ 0)	F2
Interpret ↔ compile plot parity	harness + goldens landed; residual MISMATCH tail
Runtime `timeout_seconds`	wall-clock circuit breaker on interpret
UDT `array.binary_search*` `sort_field`	interpret + compile
`Runtime.run(..., libraries=)` / `POST /run libraries`	AXIS `import ns/Name/ver` support; auto fallback keeps libs
pyne-worker 0.6.0	edge `/run`; vendored engine may lag PyPI — check versions
PyneTS (`@hoox-sh/pynets`)	standalone TS library 0.5.0 — interpret + JS compile; Python remains oracle

Component	Status
pyne-agent-worker	sister NL authoring host (`POST /v1/chat`)
Full TV platform identity	out of scope
Live plane (WS `/ws/live`)	engine-owned subscription plane with delta envelopes
Shared sessions (F-29)	refcount-pooled by content hash; SQLite journal persistence
Gevent WebSocket worker (F-30)	API default worker class
Production profile	`PYNE_DATA_DIR` pinned; Docker prod on VPS; Redis unpublished + opt-in auth

Completed themes (do not re-plan as greenfield)

- Error message / typing hygiene
- `%%pinescript` Jupyter magic + sample data helpers
- `pynescript lint`
- Yahoo / AlphaVantage / CCXT + realtime datafeed wiring
- StrategyEvent emission + parity fixtures
- Colocated TS extra tool extracted. Product TypeScript library is PyneTS.
- Alert engine + dual-host export + **L2 webhooks**
- Series caps, incremental TA (bb/kama/cmo/stochrsi), parse cache, warm compile, pending-fill, drawing GC
- Interpret dispatch inlining, unused derived OHLCV skip, incremental volume TA

Open backlog (IDs stable)

ID	Item	Pri	Status
H1	Dual-host Runtime (package SoT + backend shims + pyne-worker thin wrap; residual = worker-only extras)	P1	largely done
H2	Product warm-compile (SLOs, prewarm API/CLI, IR cache on in deploy)	P1	done
C1	Corpus set01–04 residual (parse 99.96%; Runtime 100% excl. EXPECTED)	P1	done
P1p	Compile/interpret **plot parity** residual	P1	first-party + official TV builtins **0 MISMATCH**; optional set0x unmeasured
T1	Cap `current_series` to `max_bars_back` / `_SERIES_MAX`	P2	`PYNE_SERIES_CAP` default ON
T2	Incremental TA for remaining heavy kernels	P2	volume, oscillator, and band kernels
F1	ATR Wilder / TV supertrend re-baseline only with dedicated goldens	P2	mid±factor·ATR locked; TV ratchet out of scope
F2	Pending-fill averaging when pyramiding ≤ 0	P2	done
L1	v5 ↔ v6 converter maturity	P3	done — source-level rewrites both ways (semantic bool/ `na` out of scope)
L2	Webhook alerts productization	P3	Pro + pyne-worker
L3	Legacy TS Worker builtin parity	P3	sister `hoox-sh/pine-worker` only — not a product-docs target; new TS work is PyneTS
B1	Real (non-mock) `request.*` market data	—	by design; **foreign-na policy landed**

Longer horizon (not next): ML wrappers, automatic refactor, parallel/distributed eval experiments.

Priority recommendation

Horizon	Focus
Short	H1 worker extras (logs/profile); optional set0x P1p sweep
Medium	Further nested incremental TA where profiled
Long	TypeScript library work is PyneTS (**L2** done)

Landed residual notes

- **Plot parity:** Always-on smoke in `tests/test_interp_compile_parity.py`. Full corpus compare: `python scripts/compare_interp_compile.py` → `.cache/interp_compile_parity.json`. Flags `--ignore-hline-keys` / `--ignore-fill-keys` drop structural residuals.
- **Corpus residual (C1):** set01–04 parse **99.96%** (2476/2477); Runtime interpret **100%** excl. EXPECTED_FAIL (2466 OK + 11 intentional demos); set01 **249/249**. Third-party corpora are not shipped in git.
- **Foreign-na:** `request.security` on foreign/complex expressions → `na` both backends; `ChartOHLCVProvider` refuses non-chart symbols.
- **Converter (L1):** Source-level v3 → v4 → v5 → v6 upgrades and v6 → v5 downgrades via `pynescript.util.pine_convert`; exposed as `pyne convert --to 5|6` and the `pynescript.convertToV6` LSP

command/code action. Comments and string literals are never touched; type-dependent bool/`na` migration stays out of scope.

Internals

Path	Role
`docs/ROADMAP.md`	Canonical roadmap prose
`docs/PROGRESS_REPORT.md`	Historical completion narrative
`docs/COMPILER_PLAN.md`	Compile/numba plan
`scripts/compare_interp_compile.py`	Plot parity harness
`.opencode/plans/`	Engineering plans (strategy events, consolidation)
sister `hoox-sh/pyne-worker`	Python edge evaluate host — docs
sister `hoox-sh/pyne-agent-worker`	NL authoring host — docs
`@hoox-sh/pynets`	Published **0.5.0** (interpret + JS compile); Python remains oracle

Invariants & edge cases

1. **Roadmap dates lag code** — always diff against `missing_features.md` and tests.
2. **Sister repos** (pyne-worker, pyne-agent-worker) may absorb "integration" items.
3. **AXIS is a separate product tree** (`docs/axis`) — chart UX roadmap is not PYNE-core.
4. Avoid reintroducing deleted backups (`builder.py.bak`, etc.) as "plan items."
5. Do **not** re-open **H1 / H2 / T1 / T2 / F1 / F2 / L2 / C1** as greenfield — they are marked done (H1 residual = worker extras only; P1p is corpus tail) in `docs/ROADMAP.md`.

Worked examples

Track a roadmap item with a test

```
# pick a gap, write failing test first, implement, update missing_features.md
pytest tests/test_v6_features.py -k footprint -v
```

Plot parity smoke

```
pytest tests/test_interp_compile_parity.py -q
python scripts/compare_interp_compile.py   # opt-in full compare
```

TypeScript library gate (PyneTS)

```
# standalone checkout — not present as pine-worker/ in this repo
cd ../pynets && bun test && bun run typecheck
```

See PyneTS.

Failure modes

Anti-pattern	Why it hurts
Planning features already done	Duplicated work (H2/T1/F2/L2 are done)
Roadmap without tests	Unverifiable "done"
Ignoring by-design limits	Infinite platform chase

See also

- Missing features
- Interpret ↔ compile parity
- Alerts
- pyne-worker
- pyne-agent-worker
- Contributing

CONTRIBUTING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

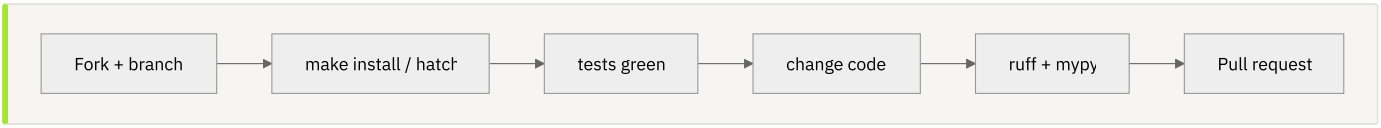
title: "Contributing" description: "Hard constraints, workflow, and style rules for contributors and agents working on PYNE."

CONTRIBUTING

Abstract

Contributions to PYNE are welcome when they respect the repo's **mechanical invariants**: generated code boundaries, src-layout packaging, dual console scripts, test corpus costs, and encryption secrets. This page distills `CONTRIBUTING.md` and `AGENTS.md` into an operational contract for humans and agents.

Conceptual model



Interface surface

Baseline workflow

1. Fork and branch from `main`
2. Install: `make install` or hatch envs
3. Run tests: `make test` / `hatch run test:test`
4. Implement with tests (prefer TDD for non-trivial behavior)
5. Lint/format: `make lint` / `make fmt` or `hatch run lint:style` + `lint:typing`
6. Open a PR with rationale + test evidence

Official short form also lives in root `CONTRIBUTING.md`.

Everyday commands

Goal	Command
Editable install + LSP	`make install`
Full tests	`make test`
LSP only	`make test-lsp`
Backend only	`make test-backend`
Lint / format	`make lint` / `make fmt`
Fast build sanity	`make build-check`
API	`make run`
LSP process	`make run-lsp`

Hard constraints

Grammar & generated code

- **Edit only** `src/pynescript/ast/grammar/antlr4/resource/*.g4` for grammar work
- **Never hand-edit** `…/antlr4/generated/` or `…/asdl/generated/`
- Regenerate via hatch `lint:gen-parser` / project scripts; see grammar guides under `.opencode/context/`

No stale backups

Do not recreate removed backups such as `builder.py.bak` or `evaluator/builtins/technical_refactored.py`. Live `builder.py` and `technical.py` are sole sources of truth.

Future annotations

Every new Python file must start with:

```
from __future__ import annotations
```

Enforced by ruff isort `required-imports`.

Console scripts

Preferred	Alias	Role
`pyne`	`pynescript`	Click CLI
`pyne-lsp`	`pynescript-lsp`	Language Server (pygls)

Import package remains `pynescript`. Do not conflate CLI and LSP entrypoints in docs, packaging, or process managers.

Optional example-script fixture

`tests/conftest.py` only expands `pinescript_filepath` when `--example-scripts-dir` points at a local directory of `*.pine` files. **No third-party corpus is shipped** in the repository.

`tests/data/library/` is reference material, not auto-parametrized.

Builtin metadata

`builtin_metadata.json` is **generated from code**. After adding builtins:

```
python scripts/generate_builtin_metadata.py
```

Do not hand-maintain the catalog long-term.

Interpret / compile parity

When changing `pynescript.runtime.Runtime`, the compiler, or plot/series export, check **series parity** between `mode="interpret"` and `mode="compile"`:

```
# Local corpus harness (default 50 scripts × 1000 bars)
python scripts/compare_interp_compile.py --bars 1000 --limit 50

# Broader pass from a path list; ignore one-sided hline/fill keys
python scripts/compare_interp_compile.py --file-list path.txt --limit 0 --workers 4 --timeout-sec 3
```

Always-on unit coverage lives in `tests/test_interp_compile_parity.py` (harness helpers + a small smoke subset). Optional longer path:

```
pytest tests/test_interp_compile_parity.py -m interp_compile_full
# or: PYNE_INTERP_COMPILE_FULL=1 pytest tests/test_interp_compile_parity.py
```

Report artifact: `.cache/interp_compile_parity.json`. Value mismatches on shared `series` keys are regressions. First-party `hline` / `fill` / `bgcolor` / `plotshape` / `barcolor` / `plotarrow` / `plotbar` / `plotcandle` keys match (0.3.12); `drawings` is geometry-only. `--ignore-hline-keys` / `--ignore-fill-keys` are optional for leftover

corpus noise. End-user mode semantics: Evaluate scripts — Runtime modes.

Secrets & crypto

- `scripts/build/.metadata.key` is **gitignored**
- CI must supply stable `CRYPTO_KEY` from `METADATA_KEY` / `_METADATA_KEY` for reproducible `.enc` blobs
- Never commit Fernet keys or API admin tokens

Internals (where to change what)

Want to...	Look at
Parse / unparse	`src/pynescript/ast/helper.py`
Add builtin	`src/pynescript/ast/evaluator/builtins/<ns>.py` + metadata generator
LSP feature	`src/pynescript/langserver/features/` + `server.py`
CLI subcommand	`src/pynescript/__main__.py`
Grammar	`resource/*.g4` then regenerate
Pro API route	`backend/api/preview.py` , `backend/app.py`
Runtime modes / compile fallback	`src/pynescript/runtime/host.py` (`Runtime.run`); `backend/runtime.py` is a shim
Interp/compile series parity	`scripts/compare_interp_compile.py` , `tests/test_interp_compile_parity.py`
TS library	sister repo hoox-sh/pynets / `@hoox-sh/pynets`

Style

- **Ruff** line length 120; broad rule set in `pyproject.toml`
- **Black** target `py310` (via toolchain config)
- **Mypy** strict-ish; exemptions for generated grammar, `evaluator.builtins.*` , tests
- Voice for docs: academic nerdy-cool per `docs/WRITING.md` — no empty marketing adjectives

Release (this repo)

Match root `CONTRIBUTING.md` and `.github/workflows/publish.yml` / `release.yml` / `ghcr.yml` :

1. Bump `__version__` in `src/pynescript/__about__.py` and update `CHANGELOG.md` .
2. Align `vscode-extension/package.json` when shipping the VSIX together.
3. Ensure CI is green on `main` .
4. Tag and push: `git tag vX.Y.Z && git push origin vX.Y.Z` .
5. **Publish** (`publish.yml`) builds sdist/wheel (`hoox_pyne-*.whl`) and uploads to PyPI (`PYPI_API_TOKEN` or Trusted Publishing OIDC, environment `pypi`).
6. **Build & Release** (`release.yml`) attaches Nuitka CLI/LSP binaries + VSIX to the GitHub Release.
7. **GHCR** (`ghcr.yml`) publishes `ghcr.io/hoox-sh/pyne/{cli,lsp,api}:X.Y.Z` on `v*` tags.

Dry-run PyPI: Actions → **Publish** → Run workflow → `dry_run=true` . Local: `python -m build && twine check dist/*` .

AXIS charting releases live in hoox-sh/axis (historical fork: `jango-blockchained/axis`).

PyneTS contributions

- Work in hoox-sh/pynets, not a `pine-worker/` directory in this repo
- Bun for install / test / typecheck; `bun run generate` after PYNE `.g4` changes
- Python remains the oracle — see PyneTS parity
- Do not copy PyneTS sources into this tree; work in the standalone hoox-sh/pynets repo / `@hoox-sh/pynets` package
- Python Runtime remains the oracle.

Failure modes for contributors

Mistake	Fallout
Editing generated ANTLR/ASDL	Overwritten / review hell
Skipping corpus cost awareness	10-minute “unit” tests
Hand-editing metadata JSON	Drift from dispatch
Changing compile/export without parity check	Silent chart/series drift vs interpret
Committing `.metadata.key`	Secret leak; rotate immediately
PR without lint	CI red on ruff/mypy

See also

- Local development
- CI
- PyneTS
- pyne-worker
- pyne-agent-worker
- Roadmap
- Root `AGENTS.md`, `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`